

Exemplu simplu de funcționare a unei rețele CNN

Scop: să vedem numeric ce face un filtru de convoluție și cum ajunge CNN-ul de la pixeli la o predicție.

1. Imaginea de intrare

Folosim o imagine alb-negru foarte mică, reprezentată printr-o matrice 4×4. Pentru simplitate, 1 = pixel activ/deschis, 0 = pixel inactiv/închis.

1	1	0	0
1	1	0	0
0	0	1	1
0	0	1	1

Într-o imagine reală, valorile ar putea fi între 0 și 255 sau normalizate între 0 și 1.

2. Filtrul (kernel-ul)

Alegem pentru demonstrație un filtru 2×2. Valorile lui sunt fixate manual doar ca să putem urmări calculul. Într-un CNN real, filtrele sunt ponderi învățate prin antrenare.

1	0
0	1

Rolul filtrului: să răspundă mai puternic în zonele imaginii care seamănă cu tiparul pe care îl reprezintă.

3. Prima operație de convoluție

În prima poziție, filtrul acoperă colțul stânga-sus al imaginii:

1	1
1	1

Calculăm produsul element-cu-element și adunăm:

$$1 \times 1 + 1 \times 0 + 1 \times 0 + 1 \times 1 = 2$$

Prima valoare din feature map este deci 2.

4. Deplasarea filtrului

Cu stride = 1, filtrul se deplasează cu un pixel spre dreapta. Următoarea regiune este:

1	0
1	0

$$1 \times 1 + 0 \times 0 + 1 \times 0 + 0 \times 1 = 1$$

La următoarea poziție avem regiunea complet zero, deci rezultatul este 0. Astfel, prima linie a feature map-ului este [2, 1, 0].

5. Feature map-ul complet

Continuând același calcul pentru toate pozițiile valide ale filtrului, obținem:

2	1	0
1	2	1
0	1	2

Feature map-ul arată unde filtrul a avut răspuns mare sau mic. O valoare mai mare înseamnă o potrivire mai puternică cu tiparul filtrului.

6. Funcția de activare ReLU

După convoluție se aplică frecvent ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

În acest exemplu toate valorile sunt deja ≥ 0 , deci feature map-ul rămâne neschimbat. Dacă ar exista, de exemplu, -2, ReLU l-ar transforma în 0.

7. Max Pooling

Pooling-ul reduce dimensiunea datelor și păstrează activările cele mai puternice. Pentru regiunea:

2	1
1	2

Max Pooling păstrează valoarea maximă:

$$\max(2, 1, 1, 2) = 2$$

În rețele reale, pooling-ul ajută la comprimarea reprezentării și la reducerea sensibilității la mici deplasări ale obiectelor.

8. Flatten

După mai multe blocuri convolution + activare + pooling, valorile pot fi transformate într-un vector. De exemplu:

$$[[2, 1], [1, 2]] \rightarrow [2, 1, 1, 2]$$

9. Stratul Dense și predicția

Vectorul intră într-un strat neuronal obișnuit. Folosim pentru demonstrație ponderile: [0.4, -0.2, 0.3, 0.5] și bias = -0.1. Aceste valori sunt din nou alese manual doar pentru calcul.

$$z = 2 \times 0.4 + 1 \times (-0.2) + 1 \times 0.3 + 2 \times 0.5 - 0.1$$

$$z = 0.8 - 0.2 + 0.3 + 1.0 - 0.1 = 1.8$$

$$\text{Aplicăm Sigmoid: } 1 / (1 + e^{(-1.8)}) = 0.8581 \approx 85.8\%$$

Dacă pragul de clasificare este 0.5, modelul atribuie observația clasei 1.

10. Ce învață de fapt CNN-ul?

Într-un CNN real, programatorul nu stabilește manual valorile filtrelor. Rețeaua pornește de la ponderi inițiale și le modifică prin backpropagation + optimizator pentru a reduce funcția Loss.

În primele straturi, filtrele tind să detecteze tipare locale simple - margini, contraste, orientări. În straturile mai profunde, combinațiile acestor tipare pot reprezenta forme și structuri mai complexe.

Schema generală



Ideea-cheie pentru masteranzi

Filtrul este un detector de tipar local. Convoluția produce un feature map, iar filtrele sunt învățate automat în timpul antrenării. CNN-ul exploatează structura spațială a imaginii, spre deosebire de un MLP care tratează intrările în principal ca un vector de caracteristici.

Observație didactică

Valorile filtrului și ale stratului Dense din acest material sunt alese manual pentru transparența calculelor. Ele nu reprezintă parametri extrași dintr-un CNN antrenat. Într-un proiect practic, următorul pas ar fi antrenarea unui CNN mic și inspectarea filtrelor efectiv învățate.